

kyagent 软件性能（核心指标）测试报告

赛题：A2「面向麒麟操作系统的安全智能运维 Agent」 作品：kyagent 文档版本：v1.0 诚实声明：性能核心指标基准整体未达标（`overall_pass = false`）。本报告按最新复测数据给出未达标项与优化方向，不夸大、不掩饰。

1. 测试目的与范围

本报告对应赛题提交要求第 5 项「软件性能（核心指标）测试报告」。目的：

1. 定义 kyagent 的核心性能指标及测量方法。
2. 给出实测数据（基于仓库实际存在的基准结果文件）。
3. 与预设目标阈值对比，明确指出哪些达标、哪些未达标，并避免把功能测试通过包装为性能达标。
4. 分析瓶颈并给出后续优化方向。

性能测试与功能测试相互独立：功能上 pytest 无失败、RealOps 8 个确定性场景全 PERFECT + 第 9 个 cron 注入对抗场景人在回路接管（见《04》）；但性能核心指标基准未通过目标阈值，本报告专门说明这一点。

2. 测试环境与方法

2.1 基准脚本

性能测试基于冻结的微基准脚本 `benchmarks/bench_ask.py`。该脚本被标注为"baseline 捕获后不得修改"，以保证基线（优化前）与当前（优化后）使用同一脚本、可对比。

运行方式：

```
# 捕获基线（一次性）
python benchmarks/bench_ask.py --save-baseline # 写 baseline.json

# 当前测量并与基线对比
python benchmarks/bench_ask.py # 写 results.json + comparison.json
```

退出码：0 表示对比整体达标，1 表示未达标。

2.2 工作负载（确定性）

`bench_ask.py` 定义固定工作负载：

- 5 次 warmup `agent.ask()`（不计时）。

- 50 次计时 `agent.ask()`：其中 30 次单工具良性查询（MockBackend 单 tool_use）、20 次多工具良性查询（ScriptedMultiToolBackend 每轮发 3 个 LOW 风险 tool_use）。
- 2000 次 `Guardrail.check_argv()` 微基准（10 种 argv × 200 次，含 `rm -rf /` 等必拒样本）。
- 1000 次 `AuditLogger.event()` 微基准。

所有指标用 `time.perf_counter_ns`（纳秒）；次级指标用 `time.process_time_ns` 记 CPU 时间。使用 MockBackend 是为了剔除真实 LLM 网络延迟，专注测量框架自身（Agent 编排 + 安全 + 审计）的开销。

2.3 测试平台

角色	文件	Python	平台
基线 baseline	benchmarks/baseline.json	3.12.10	win32
当前 results	benchmarks/results.json	3.11.13	linux
对比 comparison	benchmarks/comparison.json	—	—

说明：基线与当前测量平台不完全一致（win32 vs linux），属于框架级微基准，用于反映优化前后的相对变化；绝对值会随硬件/OS/Python 版本浮动。真实端到端时延还取决于所选 LLM 后端的网络与推理耗时（DeepSeek API），不在本框架微基准范围内。提交前如需目标平台性能证据，应在 LoongArch64 + 麒麟 V11 上重新生成 baseline/results/comparison。

3. 核心性能指标定义

指标	含义	目标阈值（相对基线）
ask p50	单次 <code>agent.ask()</code> 端到端中位延迟（框架开销，剔除真实 LLM 网络）	$\leq 0.75 \times \text{基线}$ （即中位数下降 $\geq 25\%$ ）
ask p95	<code>agent.ask()</code> 95 分位延迟	$\leq \text{基线}$ （不回退）
guardrail p50	单次 <code>Guardrail.check_argv()</code> 中位延迟	$\leq 0.75 \times \text{基线}$ （下降 $\geq 25\%$ ）
audit total	1000 次 <code>AuditLogger.event()</code> 总耗时	$\leq 0.75 \times \text{基线}$ （下降 $\geq 25\%$ ）
综合	上述四项全部达标才算 overall pass	四项 AND

辅助观测指标：`ask mean/min/max`、`total_cpu_ns`、`guardrail p95/mean`、`audit mean/event`。

4. 实测数据

4.1 基线（benchmarks/baseline.json，Python 3.12.10 / win32）

指标	数值
ask n	50

指标	数值
ask p50	9,285,600 ns $\approx$ 9.286 ms
ask p95	21,934,000 ns $\approx$ 21.934 ms
ask mean	10,167,752 ns $\approx$ 10.168 ms
ask min / max	4.986 ms / 37.708 ms
ask total_cpu	234,375,000 ns $\approx$ 234.375 ms
guardrail n	2000
guardrail p50 / p95 / mean	21,900 ns / 33,500 ns / 23,757 ns (21.90 / 33.50 / 23.76 μ s)
audit n	1000
audit total / mean	607,302,900 ns / 607,302 ns (607.30 ms / 607.30 μ s $\cdot$ event)

4.2 当前（`benchmarks/results.json`，Python 3.11.13 / linux）

指标	数值
ask n	50
ask p50	9,230,648 ns $\approx$ 9.231 ms
ask p95	15,300,004 ns $\approx$ 15.300 ms
ask mean	9,843,798 ns $\approx$ 9.844 ms
ask min / max	7.639 ms / 20.095 ms
ask total_cpu	204,431,802 ns $\approx$ 204.432 ms
guardrail n	2000
guardrail p50 / p95 / mean	2,194 ns / 3,680 ns / 2,531 ns (2.19 / 3.68 / 2.53 μ s)
audit n	1000
audit total / mean	90,851,333 ns / 90,851 ns (90.85 ms / 90.85 μ s $\cdot$ event)

5. 与目标阈值对比（`benchmarks/comparison.json`）

指标	基线	当前	ratio	delta	是否达标
ask_p50	9.286 ms	9.231 ms	0.994	-0.6%	☐ 未达标（需 $\leq$ -25%）
ask_p95	21.934 ms	15.300 ms	0.698	-30.2%	☐ 达标
guardrail_p50	21,900 ns	2,194 ns	0.100	-90.0%	☐ 达标
audit_total	607.303 ms	90.851 ms	0.150	-85.0%	☐ 达标
overall_pass	—	—	—	—	☐ false（整体未通过）

结论：性能核心指标基准整体未达标（`overall_pass = false`）。四项中 3 项（`ask_p95`、`guardrail_p50`、`audit_total`）达标且改善明显；`ask_p50` 仅小幅优于基线但未达到“下降 $\geq 25\%$ ”的目标阈值，导致综合判定不通过。我们如实报告这一结果，不将其包装为通过。

6. 瓶颈分析

`ask p50` 是端到端单轮编排的中位延迟，未达标的主要原因分析如下：

1. 编排主循环串行性：`agent.ask()` 在“LLM → 工具 → 再 LLM”多轮迭代中，工具调用与结果回填在中位路径上仍以串行为主。虽然仓库已引入并行 `tool worker` 能力（`tests/test_agent_parallel.py`、`bench_ask.py` 中对 `runtime.parallel_tools` 的探测性开关），但在 50 个样本的中位负载（含 30 个单工具查询）下，并行化对 p50 的收益有限——单工具路径本就没有可并行的多工具批次。
 2. p95 已改善但 p50 阈值很严：本轮 `ask_p95` 低于基线 30.2%，说明长尾已改善；但 `ask_p50` 只下降 0.6%，未达到“中位数下降 25%”的核心目标。
 3. 审计与守护已显著优于基线，但不是全部瓶颈：`guardrail_p50` 降 90.0%、`audit_total` 降 85.0%，说明安全裁决与审计写入已有优化效果；剩余开销集中在 Agent 编排层、消息 schema 构造、工具结果回填与计划/trace 事件路径。
 4. 测量平台差异引入噪声：基线在 win32/Py3.12、当前在 linux/Py3.11，p50 这种对常数开销敏感的指标会被运行时差异稀释，进一步压缩了“看得见的”降幅。
-

7. 后续优化方向

针对 `ask p50` 未达标，规划以下优化（按预期收益排序）：

1. 统一基线与当前的测量平台：在 LoongArch64 + 麒麟 V11、固定 Python 版本上重采基线与当前数据，消除跨平台噪声，得到可信的同环境降幅。
 2. 单工具快路径（fast-path）：对“单 `tool_use` 且 LOW 风险”的高频路径做专门短路，减少消息/schema 重复构造与逐事件审计往返。
 3. 审计批量化 / 异步落库：将 trace 事件改为批量提交或后台线程异步写 SQLite/JSONL，把审计写入彻底移出 `ask` 关键路径（当前 `audit` 已快，但仍在主线程路径上）。
 4. 工具 schema 与系统提示缓存：缓存稳定不变的 tools schema 与 system prompt 组装结果，避免每轮重复序列化。
 5. 多工具并行默认开启并扩面：把 `runtime.parallel_tools` 在安全约束下默认开启，并扩展到更多只读批次，进一步压低 p95 同时分摊 p50。
 6. 真实端到端时延单列：补充一份“真实 DeepSeek 后端端到端时延”测量（区分网络/推理/感知/执行阶段耗时），与框架微基准分开汇报，避免把网络延迟与框架开销混为一谈。
-

8. 性能测试结论

项	结论
ask p50 (核心)	未达标 (-0.6%，目标 -25%)
ask p95	达标 (-30.2%，目标不回退)
guardrail p50	达标 (-90.0%)
audit total	达标 (-85.0%)
综合 overall_pass	false (整体未通过)

总体判定：kyagent 在 ask p95、安全裁决 (guardrail) 与审计 (audit) 路径上有明显优化效果；但 agent.ask() 端到端 p50 未达到 25% 降幅目标，性能核心指标基准整体失败。我们将其作为已知局限与明确优化方向列入后续迭代，绝不以"性能已通过"作结。

复现命令： `python benchmarks/bench_ask.py` （对比基线，未达标时退出码为 1）。数据来源：
`benchmarks/baseline.json`、`benchmarks/results.json`、`benchmarks/comparison.json`。